



Machine Learning

HOMEWORK 3

NAME:

STUDENT ID#:

Objectives:

- Understand and implement a **Vision Transformer (ViT)** from scratch using PyTorch.
- Implement manual **Patch Embedding** and **Positional Encoding**.
- Build a core Transformer encoder including **Multi-Head Self-Attention (MSA)** and **Feedforward (MLP)** layers.
- Apply the model to a real-world industrial **defect detection** dataset.
- Evaluate classification performance using Confusion Matrices and class-wise visual predictions.

Part 1. Instruction

1. Vision Transformer (ViT) Implementation

In this assignment, you will implement a **Vision Transformer (ViT)** from scratch to solve an industrial image classification problem. You will use **PyTorch** for this lab, but you are **strictly forbidden from using pre-built Transformer modules** (e.g., no `torchvision.models.vit` or `timm` libraries). You must manually build the architecture by dividing images into non-overlapping patches, projecting them into embeddings, and adding learnable positional encodings. The model must include a classification head that uses the **[CLS] token** output to predict the final defect category.

2. Dataset Preparation and Training

In this assignment, you are responsible for the full data pipeline and model optimization. You must manually split the provided raw defect dataset into **70% training and 30% testing** sets before starting the training process. To ensure your model reaches high performance, you are required to **adjust the hyperparameters**, including the **learning rate, batch size, and number of epochs**. Your model must be tuned to achieve the best possible classification accuracy on the industrial samples.

3. Model Evaluation and Visualization

In this assignment, you must provide a comprehensive report that documents your model's architecture and its final performance. Your report must include specific visual evidence to verify your results:

- **Model Architecture:** A screenshot of the **model summary** showing the layers and the **total number of parameters**.

Layer (type:depth-idx)	Param #
ViT	3,264
└Sequential: 1-1	--
└└Rearrange: 2-1	--
└└└Linear: 2-2	1,088
└Dropout: 1-2	--
└Transformer: 1-3	--
└└ModuleList: 2-3	--



```

Identity: 1-4 --
Sequential: 1-5 --
  LayerNorm: 2-4 128
  Linear: 2-5 390
=====
Total params: 499,462
Trainable params: 499,462
Non-trainable params: 0
=====

```

- **Training Performance:** A screenshot of the **final training result**, clearly displaying the total epochs completed, the final test loss, and the final accuracy.

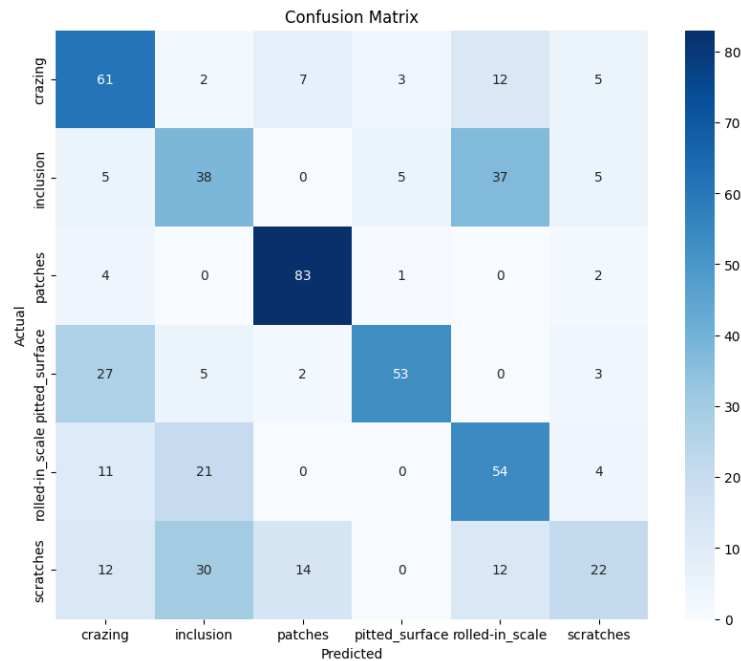
```

Epoch: 5
[ 0/60000 ( 0%)] Loss: 0.0816
[10000/60000 ( 17%)] Loss: 0.1146
[20000/60000 ( 33%)] Loss: 0.0733
[30000/60000 ( 50%)] Loss: 0.0584
[40000/60000 ( 67%)] Loss: 0.1069
[50000/60000 ( 83%)] Loss: 0.0777

Average test loss: 0.1183 Accuracy: 9603/10000 (96.03%)

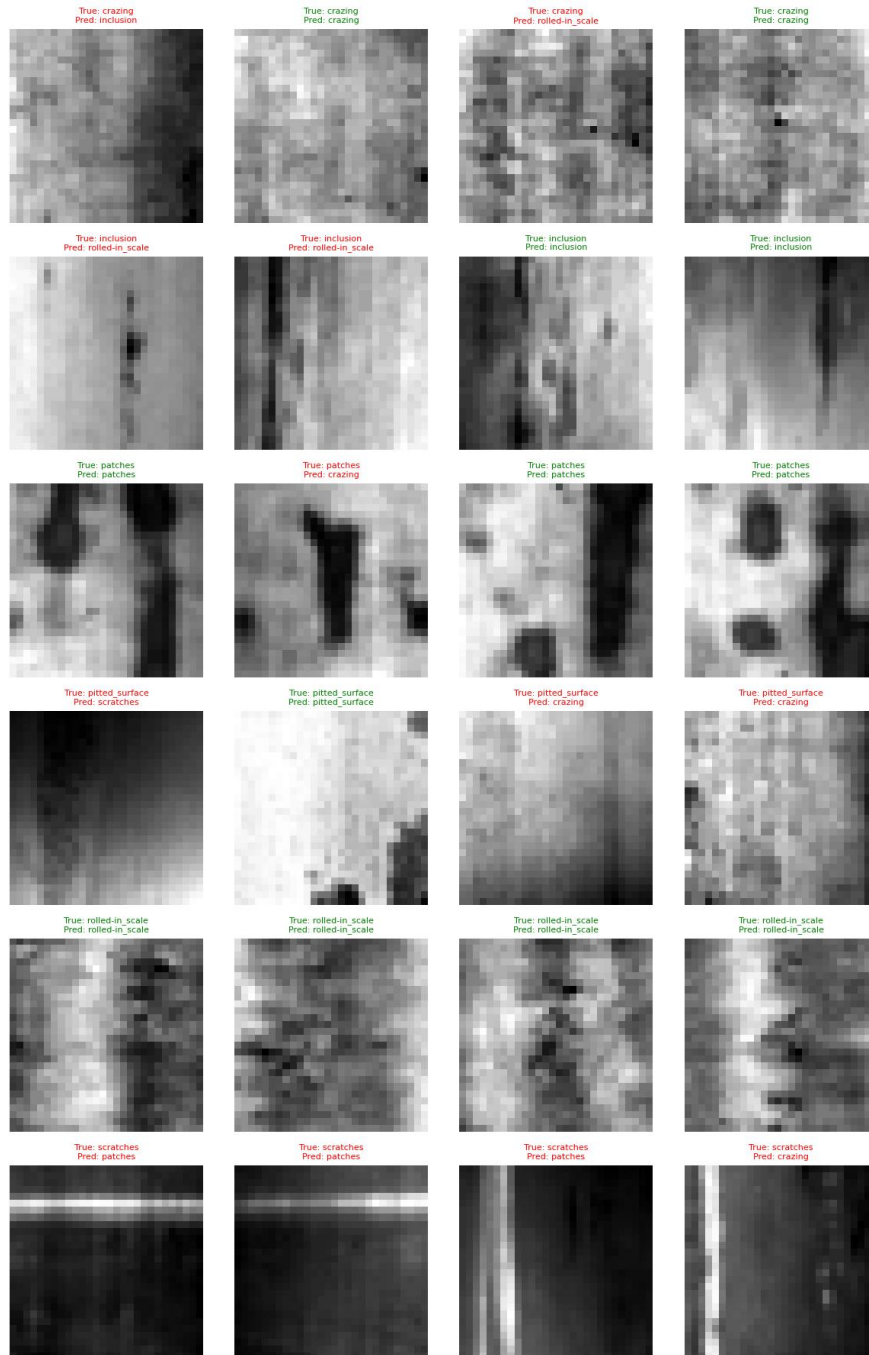
```

- **Error Analysis:** A **confusion matrix** visualizing the classification performance across all defect classes.



- **Sample Predictions:** A grid of **24 example predictions** from the test set, organized to show **4 images per class** with their true and predicted labels.





Rules:

The dataset will be provided on E3, and sample code can be found here:

<https://github.com/ahsanfs/ML-Labs>

References:

Book References: Deep Learning: Foundations and Concepts 2024th Edition.

The lecture slide Transformer for an overview of Vision Transformer design.

You should also refer to the paper: “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale” [Dosovitskiy et al., 2020, <https://arxiv.org/pdf/2010.11929>] to understand how ViT works, how to embed image patches, how positional encodings are applied, and how to design the Transformer layers.



Hint for Architecture Design: While the reference paper uses 16×16 patches for high-resolution images, our dataset consists of smaller 28×28 images. To ensure the Transformer has a sequence of patches to process, you may need to choose a smaller patch size, so that the image is divided into multiple non-overlapping patches.

Grading & Submission Instructions

Your report must present **results, figures, and analysis** based on the implementation tasks. The report should follow the outline below.

Implementation:

1. **(35%) Vision Transformer Model Design:** Implement the full ViT model from scratch, including all required components, without using any prebuilt ViT libraries; code must compile and run without critical errors.
2. **(15%) Training and Hyperparameter Tuning:** Train the model on the defect dataset and tune hyperparameters to achieve at least 80% test accuracy.
3. **(5%) Evaluation Function:** Write a correct evaluation loop to compute and report the test set's accuracy and loss.
4. **(10%) Visualization:** Provide a clear confusion matrix plot and 24 example predictions (4 per class) showing both true and predicted labels.
5. **(5%) Report Quality:** Submit a well-organized report summarizing your implementation, results, and analysis.

Question:

6. **(5%)** Briefly explain the role of patch embedding and positional encoding in ViT. You may include parts of your Step 3 model code to support your explanation.
7. **(5%)** Describe which hyperparameters you tuned, why you chose them, and how they affected your final accuracy.
8. **(5%)** Compare ViT and CNN for image classification: what are the main differences, and when might one be preferred over the other using the provided dataset?
9. **(5%)** Report the final achieved test accuracy; explain whether you reached the $>80\%$ baseline — if not, describe what you tried and why it might have failed; if you did, explain how you achieved it.
10. **(10%)** Based on the paper you referred to (Dosovitskiy et al., An Image is Worth 16×16 Words), please brief explain: *You may include parts of your Step 3 model code to support your explanation.*
 - a. How does the Vision Transformer process input images from start to finish?
 - b. How are the image patches divided and transformed into input sequences?
 - c. How does the multi-head self-attention mechanism operate within the Transformer encoder?
 - d. How does the model use the [CLS] token (or final output) to produce the final image classification?

Submission:

1. **Report:** The report must be written in English. Include screenshots of your results for each task.
2. **Code:** Submit your complete Python script (.py or .ipynb notebook).
3. Upload both your report and code to the E3 system. Name your files correctly as StudentID_HW3.zip with the following directory structure:

```

StudentID_HW3.zip
├── Transformer
│   ├── Report: StudentID_report.pdf
│   ├── Code: StudentID_code.py or StudentID_code.ipynb
│   └── StudentID.pth

```



4. ⚠ Important: Make sure all three files are uploaded. Missing even one will result in no grade for this assignment.
5. Submission Rules:
 - 1 day late: **10% deduction** from the total score.
 - Incorrect file name: **10% deduction**.
 - Incorrect directory structure: **10% deduction**.
6. Plagiarism is **strictly prohibited**. Copying reports or code from **other students, previous submissions, or any external sources will result in penalties**.
7. Contact: If you have any questions or require clarification, please email the TA (Ahsan (412540003)).

